KeyLens: see where an LLM API key is used

Build write-up · Amartya Vikram Singh · November 2026 · companion to the Guickly design doc · repo: github.com/amartya1523/keylens

> **What I built.** A working slice of the Guickly design: paste an LLM API key (Groq, xAI, OpenAI, Anthropic, OpenRouter), give each app its own virtual key, and see **which app used the key, from which client and IP, on which model, with how many tokens, at what cost**. It also shows what the traffic *would* cost off the free tier, how much rate limit is left, and **how many requests used the key without going through KeyLens at all**. Metadata only: no prompt or reply text is ever stored, and a test proves it.
> **Why.** The design doc argues that you must reconcile what the provider reports against what you observe yourself, and that coverage must be measured, not claimed. I wanted to test that with real traffic, so I built the narrowest version that touches a real provider end to end.
> **Result.** Tested live on Groq: 46 requests from 5 kinds of client across 3 apps. 9 requests were sent around KeyLens on purpose and **KeyLens counted exactly 9**, with no false ghosts on the other model. KeyLens adds about 48 ms per request. Three test suites run, and the whole build history is in small commits.

9 / 9	**5**	**76%**	**~48 ms**	**0 bytes**
outside requests detected (0 false)	client types told apart (SDKs, curl, requests, browser)	of output tokens were hidden reasoning	added per request (median, n=6)	of prompt or reply stored (canary test)

1. The question, and what a key alone can tell you

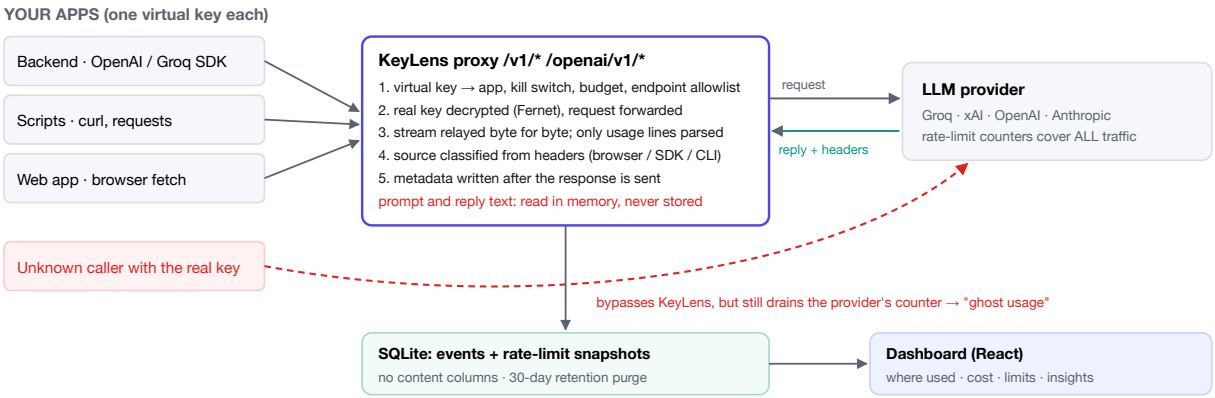
I started with a narrow question: *if all I have is an API key, can I find out where it is being used?* I checked each provider's docs and made live calls with a Groq key. The honest answer is mostly no:

Provider	What a normal key can learn about itself	Per-key usage needs
Groq	Model list only. There is no usage API (404 on every usage path). But every response carries rate-limit counters, queue and compute time, and a region code.	Not available (enterprise metrics are per project, not per key)
xAI (Grok)	`GET /v1/api-key` : name, ACLs, team, blocked flags. Responses report their own cost in `cost_in_usd_ticks` .	Management key: usage grouped by **api_key_id and request_ip**
OpenAI / Anthropic	Nothing beyond models and per-minute rate limits.	Separate admin key: usage by `api_key_id`
OpenRouter	`GET /api/v1/key` : usage, daily usage, limit, free-tier flag	n/a

No provider tells a normal key holder *where* the key was called from. So two design decisions follow, and they match the design doc's principles:

- **Be in the path, with a config change only.** Each app gets a `kl_live_…` virtual key and points its `base_url` at KeyLens. No code changes. KeyLens swaps in the real key and streams the answer back untouched.
- **Measure what you *can't* see.** Provider counters cover all traffic, ours only what we proxy. Comparing the two turns a proxy into a coverage meter and a leak detector.

2. Architecture



Backend: FastAPI + httpx (HTTP/2, streaming) + SQLite, about 1,900 lines. Frontend: React + TypeScript + Recharts, 9 pages. Prices come from a bundled snapshot of LiteLLM's public price list (732 models).

3. The three hard parts

3.1 Ghost usage: counting requests you never saw

Groq returns `x-ratelimit-limit-requests: 1000` (per day), `remaining` , and `reset-requests` , the time until the daily bucket is full again. Live check: after 1 request the reset was **86.4 s**, after 2 it was **172.8 s**. 86,400 s / 1,000 = 86.4 s. So the bucket refills one whole request every 86.4 s, and the header gives the number of requests spent recently:

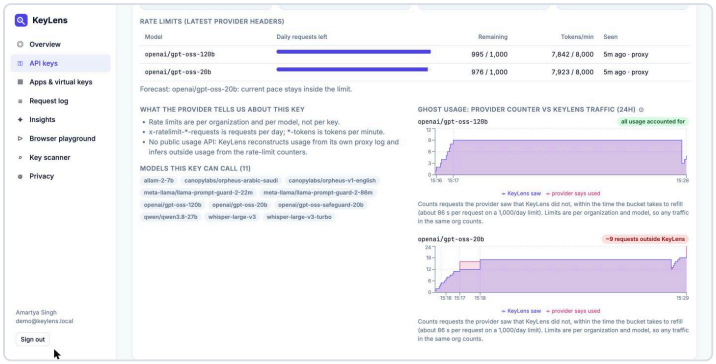
```
provider_used = reset_seconds × limit / 86400
expected = previous_reading − floor(refill since then) + requests KeyLens sent since then
ghosts += floor(provider_used − expected)  (only when the gap is ≥ 1)
```

What the first version got wrong: a continuous refill model with a fixed tolerance of 1.5. It under-counted (4 sent → 3 reported) and could invent ghosts after idle gaps, because Groq rounds the deficit up to a whole request. The review round simulated it. The fixed version re-anchors

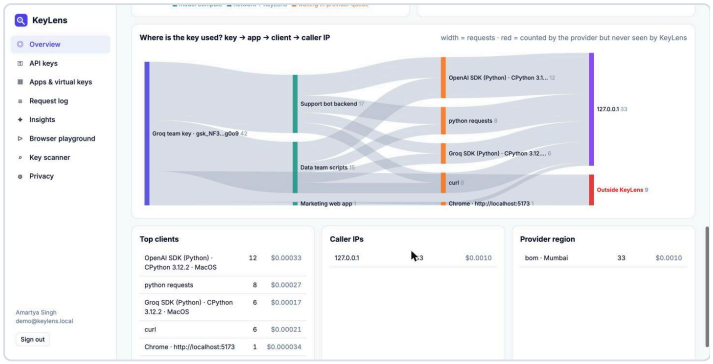
on every provider reading, credits only refills that must have happened, uses the time the headers arrived (not when the stream ended), ignores requests the provider never counted (5xx, 429), and turns itself off for per-minute windows, where an outside call refills before you can see it.

Validation	Sent around KeyLens	Reported
Live Groq, `gpt-oss-20b`, two rounds (4 + 5 direct calls)	9	**9**
Live Groq, `gpt-oss-120b` (11-minute idle gap in between)	0	**0**
Simulated bucket, 1–10 outside calls next to KeyLens traffic	n	**n (exact)**
Simulated, idle gaps 1 min–5.5 h, 4 refill phases, no outside calls	0	**0 (never over-counts)**
Simulated, outside calls before a 10-minute idle gap	2n	**2n–1 to 2n (lower bound)**

Limits are per organization and model, so the dashboard words it as "*used outside KeyLens*" and never blames a specific key. The same numbers drive a **coverage %** (seen ÷ (seen + ghost)), which is the design doc's headline metric, here measured live.



Keys page: provider counter (pink) vs what KeyLens saw (purple). The step is the 9 outside calls.



Where the key is used: key → app → client → IP, plus the red branch for traffic counted but never seen.

3.2 Where the call came from, with no content

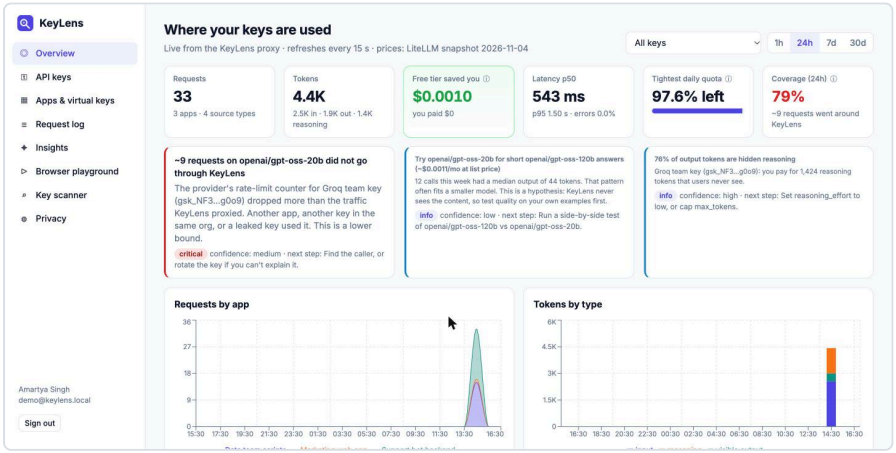
Signal (request headers only)	Label in the dashboard
`Mozilla/` UA + `Sec-Fetch-Mode` / `Origin`	Browser, e.g. "Chrome · http://localhost:5173" (host only, never the page path)
`OpenAI/Python`, `Groq/Python`, `x-stainless-runtime/os`	"OpenAI SDK (Python) · CPython 3.12.2 · MacOS"
`curl/`, `PostmanRuntime/`, `python-requests/`	CLI / API client, HTTP library

Review caught a real bug here: Node's built-in fetch also sends `Sec-Fetch-Mode`, so Node scripts were labelled "browser". The fix requires a browser user-agent as well. Labels are hints, since any client can fake a user-agent. The doc says so, and so does the UI.

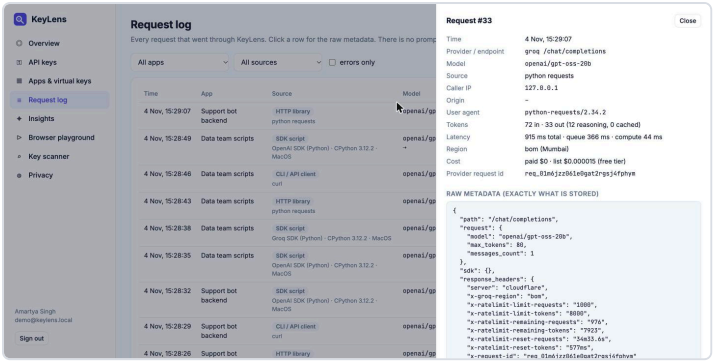
3.3 Cost that tells the truth

- **Two numbers per request:** what you actually paid ($0 on Groq's free tier, detected from its 8K TPM limit or set by hand) and the list price. The home screen leads with **"Free tier saved you $X"**.
- **Hidden reasoning tokens** are counted separately. They were 76% of output on `gpt-oss`, which becomes an insight with a fix ("set reasoning_effort low").
- **Provider edge cases:** xAI's own `cost_in_usd_ticks` wins when present · Anthropic cache reads and writes are added back into input tokens · OpenRouter `:free` models cost $0 · dated aliases resolve, but `-fast` variants are never priced as the base model.
- **Latency split:** Groq reports queue and compute time. Median queue was ~580 ms against ~50 ms compute, so the dashboard can say "your code is not the slow part".

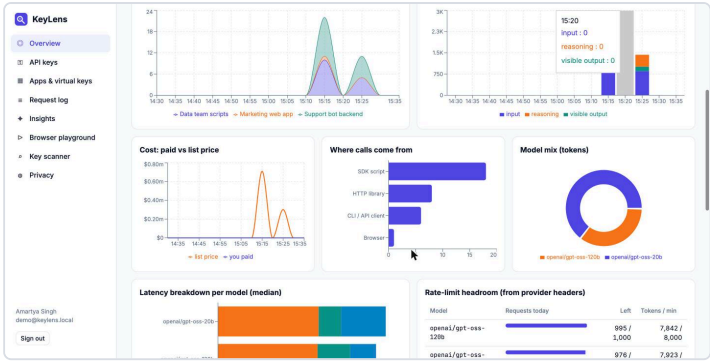
4. What it looks like



Overview on live data: requests, tokens incl. reasoning, free-tier savings, latency, tightest daily quota, and coverage (red because 9 requests went around), then insight cards with a confidence level and a next step.



Every request opens to the exact JSON that was stored: headers, usage, request parameters (numbers only). No text.



Requests by app, tokens by type, cost paid vs list, source mix, model mix.

5. What I added beyond "show me the usage"

Feature	Why it matters
Ghost usage + coverage %	The only signal here that a normal key can't get from any provider console. It catches leaked keys and unrouted apps.
Provider auto-detection	Paste any key; the provider is detected from its prefix and checked with free calls only (model list, the provider's own key-info endpoint).
Per-app kill switch and budget	Pause one app (HTTP 403) or cap it (429) without rotating the real key. Responses carry `x-keylens-budget-used`.
Quota forecast	"At this pace the daily limit runs out in ~N h", computed per model with the bucket aged forward.
Cheaper-model hypothesis	Short answers on a large model lead to a suggestion with $ saved at list price, labelled *low confidence: test quality first*, because content is never seen.
Key scanner	Finds where a key sits on disk and in shell history, whether git tracks it or has it in history, and rates the risk. Keys are only ever shown masked. It runs git with hooks disabled in untrusted repos.
Browser playground	Calls the proxy from a real browser, so the "Browser" source is demonstrated, not claimed.

6. How it maps to the design doc, honestly

Design-doc rule	Status	In the build
Metadata only; content never stored	**met**	No content columns. The canary test sends a unique prompt and reply (streaming and not) and fails if either appears anywhere in the database dump. Origins are stored as host only, and the raw metadata is capped at 4 KB.
Measured coverage, not claimed	**met**	Coverage % from ghost inference, on the home screen.
Reconcile provider data vs own observations	**met**	Rate-limit counters vs proxied traffic. Usage-API connectors (xAI management, OpenAI/Anthropic admin) are documented as the next step.
No code changes for customers	**met**	Only `base_url` and the API key change. Works with the OpenAI and Groq SDKs, curl and browsers.
Fail open; never break work	**partly**	Logging failures never break a request, and metadata is written after the response. If the KeyLens process itself is down, apps pointed at it fail. The doc's answer is a load-balancer bypass, which is not built here.
Own gateway only in v3 (in-path is the top risk)	**deliberately broken**	I built the in-path slice on purpose. It is the only slice one person can make real with live provider traffic in a few days, and it tests the riskiest claim: that metadata alone answers "where and how much". In the product I would still ship the zero-install connectors first.
Pseudonymous IDs, team floor ≥5	**not yet**	Single-user tool. Raw IPs are kept for 30 days (configurable), because the IP *is* the "where". Hashing them and team aggregation come with multi-user orgs.

7. How it was built and checked

- **Research first:** 10 parallel research passes covered each provider's docs (Groq, xAI, OpenAI/Anthropic/OpenRouter), price sources, client fingerprinting, gateway design, ghost math, the scanner and the UX, plus live probes of the Groq key. Notes are in `docs/research.md`.
- **Then 5 independent reviews** (backend correctness, security, frontend, ghost math, an interviewer's read) found real issues, all fixed: a metering bypass via an encoded `%23models` path (now an endpoint allowlist) · DNS rebinding against the local API (trusted hosts) · cross-user queries missing `user_id` · Anthropic cache pricing · stream aborts losing events · gzip handling · scientific notation in tiny costs · the ghost model itself.
- **Tests:** `test_privacy.py` (canary text never stored) · `test_proxy_rules.py` (allowlist, kill switch, budget, bad keys, odd bodies, Host check) · `test_ghost.py` (never over-counts, exact for close readings).
- **Measured overhead:** about 100 ms added at first. Writing metadata after the response and caching the cipher brought it to about 48 ms. The next step is caching virtual-key lookups.

8. Limits and next steps

- Only Groq was tested with a live key. The xAI, OpenAI, Anthropic and OpenRouter code paths follow their docs but were not called for real.

- Ghost inference needs a long request window (Groq's daily limit). Outside calls are only visible within the refill time (~86 s per request on a 1,000/day limit), so it is a lower bound by design.
- **Next:** usage-API connectors (xAI groups by key and IP, so per-IP history works even before KeyLens existed) · multi-user orgs with identity resolution and pseudonymous IDs · a fail-open bypass · cached key lookups.

Run it: `cd backend && pip install -r requirements.txt && uvicorn app.main:app` · `cd frontend && npm install && npm run dev` · sign up → paste a key → create an app → call `http://localhost:8000/v1` with its virtual key.